

BANK ACCOUNT MANAGEMENT

Java Object-Oriented Programming Experiment

Experiment	Bank Account Management using Inheritance
Language	Java
Main Class	BankAccountManagement

1. Aim / Objective

To develop a Java program for managing savings and current bank accounts using classes, inheritance, constructors, method overriding, the **this** keyword, the **super** keyword, and account operations such as deposit, withdrawal, interest calculation, and money transfer.

2. Description

The program defines a base Account class containing account number, balance, and account type. SavingsAccount extends Account and calculates interest. CurrentAccount extends Account and overrides withdraw() to support an overdraft limit. The main class creates both accounts and performs the operations.

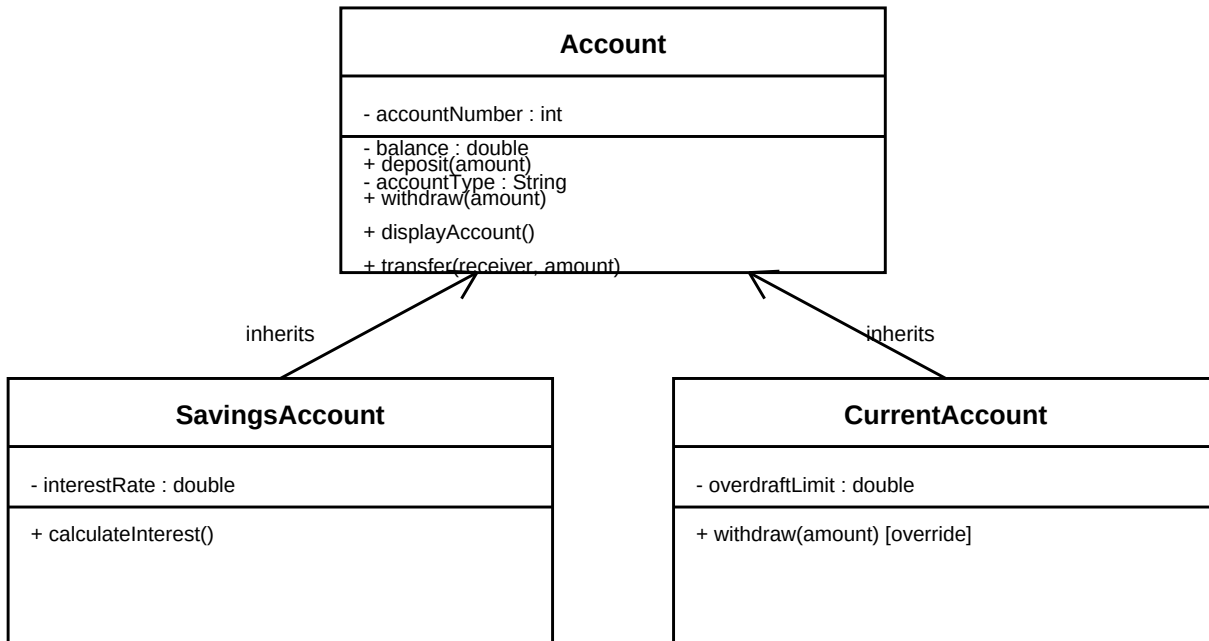
3. Software Required and Hardware Required

Category	Requirement
Software	Java Development Kit (JDK 8 or later)
IDE	Eclipse IDE (or any Java IDE)
Operating System	Windows / Linux / macOS
Hardware	Computer or laptop
Processor	1 GHz or higher
RAM	2 GB minimum (4 GB recommended)
Storage	At least 500 MB free space

4. Algorithm

1. Start the program and create a Scanner object.
2. Create a SavingsAccount with account number 101, balance Rs.10000, and interest rate 5%.
3. Create a CurrentAccount with account number 102, balance Rs.5000, and overdraft limit Rs.3000.
4. Display savings account details.
5. Deposit Rs.2000 and withdraw Rs.1500 from the savings account.
6. Calculate and add 5% interest.
7. Display the updated savings account.
8. Display the current account, deposit Rs.3000, and withdraw Rs.9000 using the overdraft facility.
9. Transfer Rs.1000 from SavingsAccount to CurrentAccount.
10. Display both final account details.
11. Close Scanner and stop the program.

5. UML Diagram



SavingsAccount and CurrentAccount inherit from Account. CurrentAccount overrides withdraw(), while SavingsAccount adds calculateInterest().

6. Source Code

```
import java.util.Scanner;

class Account {
    int accountNumber;
    double balance;
    String accountType;

    Account(int accountNumber, double balance, String accountType) {
        this.accountNumber = accountNumber;
        this.balance = balance;
        this.accountType = accountType;
    }

    void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Amount deposited successfully.");
    }

    void withdraw(double amount) {
        if (amount <= balance) {
            balance = balance - amount;
            System.out.println("Amount withdrawn successfully.");
        } else {
            System.out.println("Insufficient balance.");
        }
    }

    void displayAccount() {
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Account Type : " + accountType);
        System.out.println("Balance : " + balance);
    }

    void transfer(Account receiver, double amount) {
        if (amount <= balance) {
            balance = balance - amount;
            receiver.balance = receiver.balance + amount;
            System.out.println("Amount transferred successfully.");
        } else {
            System.out.println("Insufficient balance for transfer.");
        }
    }
}

// Savings Account
class SavingsAccount extends Account {
    double interestRate;

    SavingsAccount(int accountNumber, double balance,
        String accountType, double interestRate) {
        super(accountNumber, balance, accountType);
        this.interestRate = interestRate;
    }
}
```

```

    }

    void calculateInterest() {
        double interest = balance * interestRate / 100;
        balance = balance + interest;

        System.out.println("Interest Rate : " + interestRate + "%");
        System.out.println("Interest Added: " + interest);
    }
}

// Current Account
class CurrentAccount extends Account {
    double overdraftLimit;

    CurrentAccount(int accountNumber, double balance,
        String accountType, double overdraftLimit) {

        super(accountNumber, balance, accountType);
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    void withdraw(double amount) {
        if (amount <= balance + overdraftLimit) {
            balance = balance - amount;
            System.out.println("Amount withdrawn successfully.");
        } else {
            System.out.println("Overdraft limit exceeded.");
        }
    }
}

// Main Class
public class BankAccountManagement {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Creating Savings Account
        SavingsAccount savings =
            new SavingsAccount(101, 10000, "Savings", 5);

        // Creating Current Account
        CurrentAccount current =
            new CurrentAccount(102, 5000, "Current", 3000);

        System.out.println("===== SAVINGS ACCOUNT =====");

        savings.displayAccount();

        System.out.println("\nDepositing Rs.2000");
        savings.deposit(2000);

        System.out.println("\nWithdrawing Rs.1500");
        savings.withdraw(1500);

        System.out.println("\nCalculating Interest");
        savings.calculateInterest();

        System.out.println("\nUpdated Savings Account:");
        savings.displayAccount();

        System.out.println("\n===== CURRENT ACCOUNT =====");

        current.displayAccount();

        System.out.println("\nDepositing Rs.3000");
        current.deposit(3000);

        System.out.println("\nWithdrawing Rs.9000");
        current.withdraw(9000);

        System.out.println("\nUpdated Current Account:");
        current.displayAccount();

        System.out.println("\n===== TRANSFER =====");

        System.out.println("Transferring Rs.1000 from Savings to Current");

        savings.transfer(current, 1000);

        System.out.println("\nSavings Account after transfer:");
        savings.displayAccount();

        System.out.println("\nCurrent Account after transfer:");
        current.displayAccount();

        sc.close();
    }
}

```

7. Compilation and Execution Commands

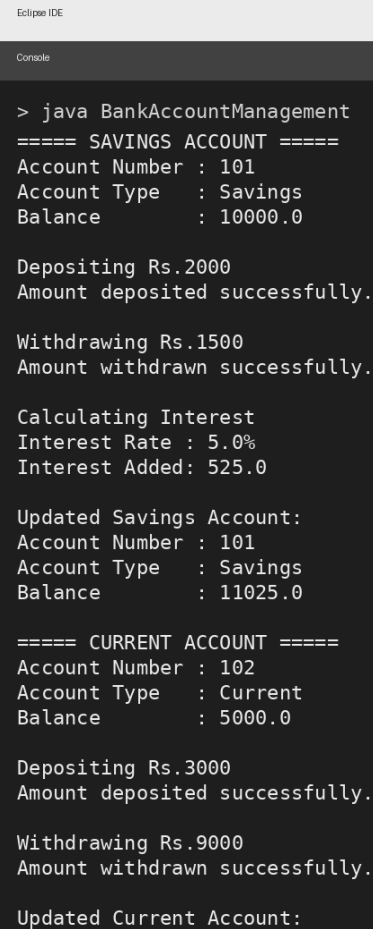
Save the source file as **BankAccountManagement.java** and run:

```
javac BankAccountManagement.java
java BankAccountManagement
```

In Eclipse: Run → Run As → Java Application. The output appears in the Eclipse Console.

8. Output Screenshot

Eclipse-style console output:



```
Eclipse IDE
Console

> java BankAccountManagement
===== SAVINGS ACCOUNT =====
Account Number : 101
Account Type   : Savings
Balance       : 10000.0

Depositing Rs.2000
Amount deposited successfully.

Withdrawing Rs.1500
Amount withdrawn successfully.

Calculating Interest
Interest Rate : 5.0%
Interest Added: 525.0

Updated Savings Account:
Account Number : 101
Account Type   : Savings
Balance       : 11025.0

===== CURRENT ACCOUNT =====
Account Number : 102
Account Type   : Current
Balance       : 5000.0

Depositing Rs.3000
Amount deposited successfully.

Withdrawing Rs.9000
Amount withdrawn successfully.

Updated Current Account:
```

9. Result

Thus, the Bank Account Management program was successfully implemented and executed using Java. It demonstrates inheritance, constructor chaining using super, this, method overriding, deposit, withdrawal, interest calculation, overdraft handling, and account transfer. The final savings balance is Rs.11025.0 and the final current account balance is Rs.0.0.